

i9 Smart ERP Tutorial

Exportado a partir de deploy-checklist.md

i9 Smart ERP — Checklist de Deploy e Teste

■■■ PASSO 1: Supabase (banco de dados)

1. Acesse <https://supabase.com> e crie um projeto
2. Copie: **Project URL** e **service_role key**
3. Preencha o `.env`
4. Rode:

```
npm run setup
```
5. Verifique no painel: **Table Editor** deve mostrar as tabelas principais e os dados de teste provisionados

■■■ PASSO 2: Variáveis de ambiente

```
cp .env.example .env
```

Edite `.env`:

```
SUPABASE_URL=https://SEU-PROJETO.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SERVICE_ROLE_KEY=eyJ... # alias opcional para compatibilidade
DATABASE_URL=postgresql://postgres:SENHA@db.SEU-PROJETO.supabase.co:5432/postgres
PORT=4000
NODE_ENV=production
```

■■■ PASSO 3: Backend

```
npm install
npx prisma generate
npm run db:bootstrap # sincroniza modelos e provisiona o ambiente de teste
npm run build        # compila TS para dist/

# Desenvolvimento
npm run dev          # inicia os dois servidores da API

# Verificar saúde
curl http://localhost:4000/health
# Esperado: {"status":"ok","version":"5.0.8"}
```

■■■ PASSO 4: Testes dos endpoints

Login

```
curl -X POST http://localhost:4000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"gestor@arteemovimento.com","password":"Gestor123"}'
# Guarde o access_token retornado como $TOKEN
```

Registrar ponto (timestamp automático no servidor)

```
curl -X POST http://localhost:4000/ponto/registrar \
```

```
-H &quot;Authorization: Bearer $TOKEN&quot; \
-H &quot;Content-Type: application/json&quot; \
-d &#x27;{&quot;tipo&quot;:&quot;entrada&quot;,&quot;latitude&quot;:-29.6868,&quot;longitude&quot;:
```

Fazer chamada (data/hora automáticos — professor não informa)

```
curl -X POST &quot;http://localhost:4000/turmas/t0000000-0000-0000-0000-000000000001/chamada&quot; \
-H &quot;Authorization: Bearer $TOKEN&quot; \
-H &quot;Content-Type: application/json&quot; \
-d &#x27;{
  &quot;topico&quot;: &quot;Variações de Barra&quot;,
  &quot;registros&quot;: [
    {&quot;aluno_id&quot;: &quot;al000000-0000-0000-0000-000000000001&quot;, &quot;presente&quot;:
    {&quot;aluno_id&quot;: &quot;al000000-0000-0000-0000-000000000002&quot;, &quot;presente&quot;:
  ]
}&#x27;
```

Gerar convite com link instalador

```
curl -X POST http://localhost:4000/convites \
-H &quot;Authorization: Bearer $TOKEN&quot; \
-H &quot;Content-Type: application/json&quot; \
-d &#x27;{
  &quot;role_destino&quot;: &quot;professor&quot;,
  &quot;email_destino&quot;: &quot;novo@email.com&quot;,
  &quot;nome_destino&quot;: &quot;Novo Professor&quot;,
  &quot;metadata&quot;: {&quot;turma_id&quot;: &quot;t0000000-0000-0000-0000-000000000001&quot;};
}&#x27;
# Retorna: {&quot;link&quot;: &quot;https://app.i9smart.com.br/instalar/professor?token=UUID&quot;}
```

PGR — Registro de humor (APPEND-ONLY)

```
curl -X POST http://localhost:4001/pgr/humor \
-H &quot;Authorization: Bearer $TOKEN&quot; \
-H &quot;Content-Type: application/json&quot; \
-d &#x27;{&quot;humor&quot;:&quot;bem&quot;,&quot;quer_contato&quot;:false,&quot;anonimo&quot;:false}
```

Avaliação de aula pelo aluno (UPSERT — editável)

```
curl -X POST &quot;http://localhost:4001/avaliacoes/aula/au000000-0000-0000-0000-000000000001&quot; \
-H &quot;Authorization: Bearer $TOKEN&quot; \
-H &quot;Content-Type: application/json&quot; \
-d &#x27;{&quot;avaliacao&quot;:&quot;feliz&quot;,&quot;comentario&quot;:&quot;Aula ótima!&quot;}&#x27;
```

Notificar inadimplência (cron manual)

```
curl -X POST http://localhost:4001/cron/verificar-inadimplencia \
-H &quot;Authorization: Bearer $TOKEN_SUPORTE&quot;
```

■■■ PASSO 5: App (frontend standalone)



```
# Testar localmente — abrir no browser
open app-v3.html

# Produção — Vercel
npx vercel --prod

# Ou arrastar app-v3.html para netlify.com/drop
```

Seletor de portais (demo): canto superior esquerdo da tela.

Em produção, cada perfil recebe seu próprio link de convite com o app instalador.

Painel de diagnóstico: abra `backend-test.html` para validar login, health e rotas por perfil sem depender dos portais.

■■■ PASSO 6: Criar senhas no Supabase Auth



1. Acesse o painel: **Authentication** → **Users**

ou use a API Admin:

<https://www.pearsoncmg.com/api/v1/print/healthcare/9780134477223>

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved.

— — —

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10984-2135. Printed in the United States of America. This publication is protected by copyright. Permission to reproduce copies may be obtained from Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116. For more information, contact your local Pearson Education representative or write to Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116. ISBN 0-203-01177-9

PGR / NR-1

- [] POST /pgr/humor retorna 201
- [] DELETE em pgr_emotional_logs falha com erro do trigger
- [] Dashboard do gestor mostra dados anônimos
- [] Relato urgente gera notificação para gestor

Planos de aula

- [] Gestor cria plano com status "bloqueado"
- [] Professor não vê planos bloqueados
- [] Gestor publica → professor vê
- [] Aluno vê atividades das aulas publicadas

Financeiro

- [] Pix Cópia e Cola gerado corretamente
- [] Responsável recebe notificação após 5 dias de atraso
- [] Gestor marca pago em dinheiro → status atualiza

Logo

- [] Suporte faz upload (JPG/PNG)
- [] Logo aparece no header de todos os portais
- [] Outros perfis não têm a opção de upload

Remoção

- [] Gestor remove usuário → acesso bloqueado imediatamente
- [] Somente suporte remove escola
- [] Escola removida tem 30 dias para BKP

powered by i9 Smart · Versão: 5.0.8 (10013073)